Computer Interfacing (CI) FIB. Primer parcial **03/11/2025**

Cognoms, Nom _____SOLUCIÓ_____ DNI _____

Tota resposta sense justificar es considerarà nul·la !

P1. (2 punts) Tenim aquesta funció "CLR_BNK" que esborra un Bank de la memòria de dades. Recordeu que el nostre model té 1,5 KB de RAM disponibles (General Purpose Registers). La funció rep un paràmetre pel WREG que indica el Bank a borrar. Retorna 0 si no hi ha hagut cap problema.

```
; Funció CLR_BNK
CLR_BNK:  SUBLW   5
          BNN     OK
          RETLW   -1
OK:       SUBLW   5
          MOVWF   FSR0H, A
          CLRF    FSR0L, A
BUCLE:    CLRF    POSTINC0
          TSTFSZ  FSR0L, A
          BRA     BUCLE
          RETLW   0
; Final Funció
```

```
;Exemple 1 de crida (esborrar el Bank 2):
          MOVLW   2
          CALL    CLR_BNK

;Exemple 2 de crida (esborrar el Bank 7):
          MOVLW   7
          CALL    CLR_BNK
```

SUBLW	**Subtract W from literal**
Syntax:	SUBLW k
Operands:	$0 \le k \le 255$
Operation:	$k - (W) \rightarrow W$
Status Affected:	N, OV, C, DC, Z
Encoding:	0000 | 1000 | kkkk | kkkk
Description	W is subtracted from the 8-bit literal 'k'. The result is placed in W.

1.1 (0,25p) Quants Bytes ocupa el codi de la funció a memòria de programa?

Totes les instruccions són d'una paraula a memòria, 1 word = 2 bytes, per tant:
2B (SUBLW) + 2B (BNN) +2B (RETLW) +2B (SUBLW) + 2B (MOVWF) +2B (CLRF) + 2B (CLRF) + 2B (TSTFSZ) + 2B (BRA) + 2B (RETLW) = **20 Bytes**

1.2 (0,75p) Amb el Clock del Processador a 8 MHz, quant temps triga a executar-se si li passem el paràmetre 2, com a l'exemple 1? (Per l'execució, compteu el MOVLW i el CALL com a part de la funció)

```
          MOVLW   2          → Wreg=2                    → 1 cicle
          CALL    CLR_BNK    → Entra a la funció         → 2 cicles
CLR_BNK:  SUBLW   5          → Wreg = 5 - 2 = 3          → 1 cicle
          BNN     OK         → Salta a OK                → 2 cicles
          RETLW   -1
OK:       SUBLW   5          → Wreg = 5 - 3 = 2          → 1 cicle
          MOVWF   FSR0H, A                               → 1 cicle
          CLRF    FSR0L, A   → FSR0 = 200h               → 1 cicle
BUCLE:    CLRF    POSTINC0   → Borra i incrementa FSR0   → 1 cicle els 256 cops
          TSTFSZ  FSR0L, A   → Espera a arribar a 300h   → 255 cops 1 cicle, 1 2 cicles
          BRA     BUCLE      → Salta a bucle             → 255 cops 2 cicles (saltar)
          RETLW   0          → Surt                      → 2 cicles
```

Per tant necessita : 9 cicles + 255 (1 + 1 + 2) + 1 (1 + 2) + 2 cicles = **1034 cicles d'instrucció**
Amb el clock a 8 MHz, vol dir Tosc = 125ns, Tinst = 500 ns

Per tant el codi triga: 1034 cicles d'instrucció * 500 ns = **517 us** (**517 microsegons)**

1.3 (0,25p) I si li passem el paràmetre 7 al WREG (exemple 2)?

```
            MOVLW   7           → Wreg=7                    → 1 cicle
            CALL    CLR_BNK     → Entra a la funció         → 2 cicles
CLR_BNK:    SUBLW   5           → Wreg = 5 - 7 = -2         → 1 cicle
            BNN     OK          → No Salta a OK (negatiu)   → 1 cicle
            RETLW   -1          → Surt de la funció         → 2 cicles
OK:         SUBLW   5
            MOVWF   FSR0H, A
            CLRF    FSR0L, A
BUCLE:      CLRF    POSTINC0
            TSTFSZ  FSR0L, A
            BRA     BUCLE
            RETLW   0
```

Per tant necessita : 1 + 2 + 1 + 1 + 2 cicles = **7 cicles d'instrucció**
Amb el clock a 8 MHz, vol dir Tosc = 125ns, Tinst = 500 ns

Per tant el codi triga: 7 cicles d'instrucció * 500 ns = **3,5 us** (**3,5 microsegons)**

1.4 (0,5p) Indica, si es pot saber, amb quin valor acaben els registres FSR0H i FSR0L en els dos casos anteriors.

En el cas MOVLW 2, FSR0H acaba 3, FSR0L acaba a 00 (FSR0=300h)
En el cas MOVLW 7, no sabem amb quins valors acaben ja que no els modifiquem (tindran els previs).

1.5 (0,25p) Si en comptes del PIC 18F45K22 que té 1536 Bytes de GPR-RAM, féssim servir el 18F44K22 que només té 768 Bytes, què hauria de canviar a la funció?

El codi intenta evitar que borrem un Bank de memòria RAM (General Purpose Registers) que no existeix.
El nostre PIC té 1536 Bytes de GPR, 1536/256 = 6 Banks, per això admetem valors entre 0 i 5.

Si tenim 768 Bytes, vol dir que tindrem 3 Banks, i admentrem valors entre 0 i 2.
Per tant canviarem dues línies:

```
CLR_BNK:    SUBLW    2
            BNN      OK
            RETLW    -1
OK:         SUBLW    2
            …
```

Computer Interfacing (CI) FIB. Primer parcial **03/11/2025**

Cognoms, Nom _____SOLUCIÓ_____ DNI _____

Tota resposta sense justificar es considerarà nul·la !

P2. (1,5 punts) La interrupció PORTB CHANGE es dispara quan hi ha un canvi en algun (alguns) dels 4 bits alts del PortB <7:4>. El símbol corresponent a les interrupcions pel canvi del PORTB és RB. A continuació veureu un codi (main i RSI) que un estudiant ha fet per saber què li ha passat al PortB, amb la idea de que la variable "Motiu" li indiqui quin bit ha canviat.

```
BYTE Motiu, B_Actual, B_Anterior, B_Bits;

void main(void)
{
   ANSELB = ANSELB & 0x0F;
   TRISB = TRISB | 0xF0;
   RBIE=1;
   RBIP=1;
   RBIF=0;
   IPEN=1;
   B_Anterior=PORTB;
   Motiu=0;
   GIEH=1;
   …..// Resta del main
}
```

```
void interrupt RSI()
{
   if ((RBIF==1) && (RBIE==1))
   {
      B_Actual = PORTB;
      B_Bits= (B_Anterior ^ B_Actual) & 0xF0;
      B_Anterior=B_Actual;
      if (B_Bits & 0x80) Motiu=7;
      if (B_Bits & 0x40) Motiu=6;
      if (B_Bits & 0x20) Motiu=5;
      if (B_Bits & 0x10) Motiu=4;
   }
}
```

2.1 (0,5p) Si el PORTB tenia un valor inicial de 0x00 (tots els pins a 0V) i en un moment donat canvia al valor 0x31, es disparará la interrupció associada a RB ? Quin valor tindrà la variable Motiu quan s'hagi executat la RSI ?

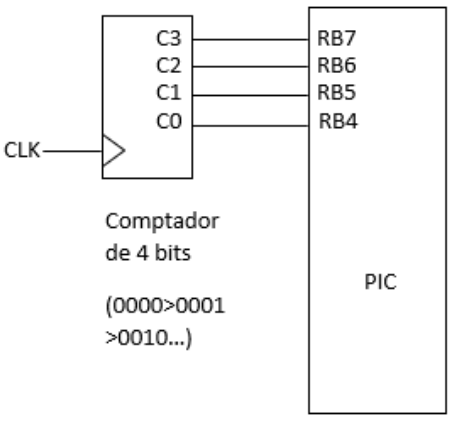
Si, canviarà perquè els bits alts passen de **0000 (del 0x00) a 0011 (del 0x31)**.
La variable **Motiu acabarà a 4**, perquè s'haurà evaluat dos cops, primer a 5 (bit 5) i després a 4 (bit 4). Al canviar dos bits, el segon if sobreescriu el primer.

2.2 (0,5p) L'estudiant ha executat el programa i no funciona com esperava. Què ha fet malament??

A banda de sobreescriure la variable Motiu, que s'hauria de corregir, hi ha un problema molt més greu.

Dins de la RSI **no borra el Flag de RBIF** que l'ha fet entrar a la interrupció. Per tant, entrarà constantment a la interrupció i no executarà mai més res del "main".

2.3 (0,5p) Un cop ha arreglat el problema exposat a 2.2, connecta un comptador de 4 bits als pins RB4-RB7, com es veu a la figura. Si el CLK posat al comptador és de 1 MHz i Fosc al PIC és de 16 MHz, podrem detectar tots els canvis dels bits del comptador al PORTB? Per què?

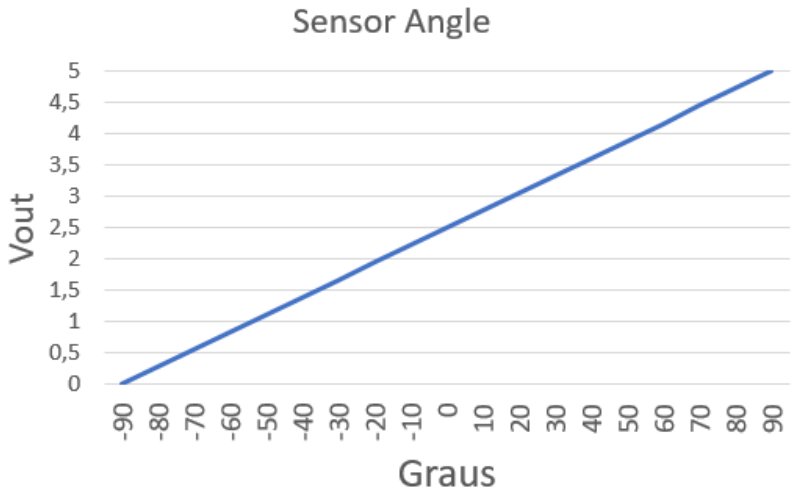


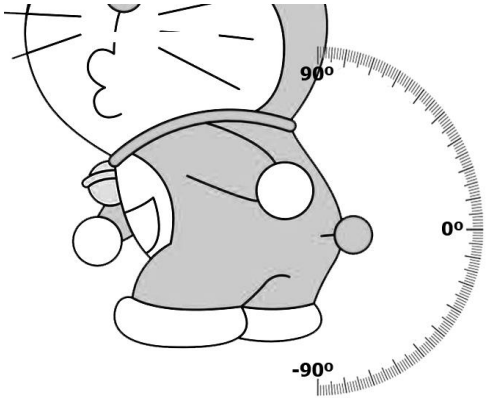
El PIC amb Fosc=16MHz, tindrà un cicle d'instrucció de 250ns.

Estat inicial 0000
1us
Estat 0001
1us
Estat 0010
etc.

Com podem veure, cada 1 us hi haurà canvis als bits del PORTB. El temps de **Latència** de la interrupció serà 3 o 4 cicles (750 a 1000 ns) i el codi de la RSI necessitarà unes quantes **instruccions**, per tant **no podrem detectar tots els canvis al comptador.**

P3. (2 punts) Un conegut gat còsmic del futur ens ha demanat ajuda per arreglar una de les parts del seu cos robòtic: El sensor d'inclinació de la seva cua. Aquest sensor mesura l'angle d'inclinació de la cua entre -90º i 90º. Podeu trobar una caracterització de la seva sortida a continuació:





(1p) Ho digitalitzarem amb l'AD d'un PIC18F45K22. Si Fosc=10MHz, quantes mostres per segon podeu arribar a aconseguir? Suposeu TAD>=1us i Tacq>=7.45us

La millor configuració que podem obtenir i que compleix les restriccions de l'enunciat és:
TAD=16/Fosc -> TAD=1,6us, ja que les que fan servir un divisor de freqüència més petit ens fan obtenir TADs < 1us
Amb un TAD de 1,6us, el millor temps d'adquisició que podem aconseguir un **Tac =6*TAD= 9,6us**.
Sabem que per obtenir una mostra ho d'AD ho podem aproximar com Tmostra=Tacq+12*TAD que si ho calculem és **Tmostra=28,8us**

Podem aconseguir 1/28,8= **34722,2 mostres en un segon**.

(0,5p) Ara volem fer servir també el mateix PIC per digitalitzar el so que fa el picarol del nostre amic del futur. Indica quina és la freqüència màxima que pot tenir el so si volem evitar l'aliasing i no deixem de mesurar la inclinació de la cua (a la mateixa freqüència de mostreig que el picarol). Nota: segons Nyquist, per evitar l'aliasing la freqüència de mostreig ha de ser major que el doble de la freqüència del senyal.

Al nostre PIC tenim un únic AD amb diverses entrades multiplexades. Si ara afegim una nova mesura, sense deixar de mostrejar l'anterior vol dir que el temps serà compartit entre les dos mesures, multiplexant-se en el temps: Mesura inclinació cua, mesura f picarol, Mesura inclinació cua, mesura f picarol…
En conseqüència podem obtenir mostres del picarol (fent servir el resultat de l'apartat anterior) a 17361,1Hz i segons Nyquist la **freqüència màxima del picarol no pot ser superior a 8680,5Hz** si volem evitar l'aliasing.

(0,5p) Si volem una resolució de 0.05 graus, quants bits d'AD necessitarem? Suposeu que Vref+=5V i Vref-=0V

Veiem que el rang del nostre AD és de 0V a 5V, que es corresponen a inclinacions entre -90º i 90º (180º de rang)
La resolució que busquem és $0.05º = 180/2^N - 1$. Resolem N i trobem que és 11,81 bits i podem concloure que **necessitem 12bits** per obtenir la resolució desitjada.

P4. (1 punt) Respecte a les pràctiques de laboratori, contesta breument en l'espai assignat a les següents preguntes:

Computer Interfacing (CI) FIB. Primer parcial **03/11/2025**

Cognoms, Nom _____SOLUCIÓ_____ DNI _____

Tota resposta sense justificar es considerarà nul·la !

(0,2p) Quina és la freqüència Fosc a la que treballa el PIC18 de l'easyPIC? A on configures a Proteus que el micro funcioni a aquesta Fosc?

Treballa a 8MHz. A proteus es configura a les propietats del PIC a l'esquemàtic, fent clic dret i canviant la propietat "Processor Clock Frequency"

(0,2p) Quina diferència hi ha entre compilar en mode Debug i en mode Release a Proteus?

Al mode debug es genera un arxiu .cof, que permet debugar el nostre codi, per exemple posant breakpoints, executant pas a pas o mirant el contingut de la memòria del nostre micro. En mode release es genera un fitxer .hex que permet ser carregat a la memòria de programa del micro de la EasyPIC

(0,2p) Per què cal afegir una resistència en sèrie amb un LED quan el volem encendre amb un pin del Micro?

Cal limitar la corrent que arriba al LED per tal de que emeti la llum que ens interessa i protegir els components, el pin del PIC i el propi LED, de sobrecorrents

(0,2p) En el codi de la pràctica, per què és necessari afegir #define _XTAL_FREQ 8000000?

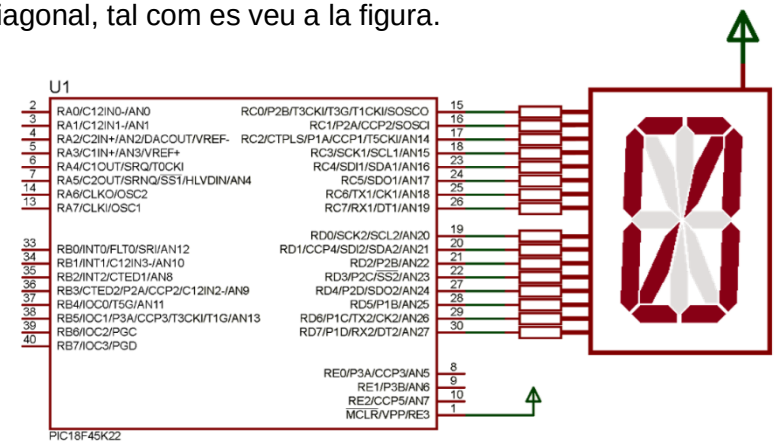
Això no configura la freqüència del microcontrolador. El que fa és informar al compilador de quina és la freqüència amb què realment funciona el PIC, perquè així les funcions delay funcionin correctament. Aquestes funcions necessiten saber la freqüència d'oscil·lació per calcular quants cicles de CPU han de passar per generar el retard exacte.

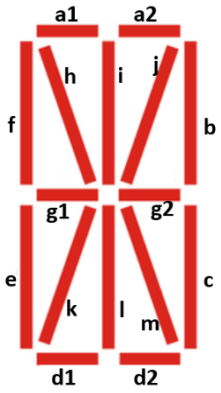
(0,2p) Explica per què és important no parar de refrescar els 7-segments a la easyPIC quan volem mostrar quatre dígits

Quan es treballa amb una placa com EasyPIC i es vol mostrar quatre dígits en un display de 7 segments multiplexat, és essencial no deixar mai de refrescar-los. Això és degut a com funciona la multiplexació. Només podem mostrar una xifra a la vegada, per exemple podem mostrar un 7 o un 77 o un 777, però no un 1234 al mateix instant ja que els displays de 4 dígits comparteixen tots els segments (a, b, c, d, e, f, g, dp). Només tenen separat el dígit que activa cada display, encenent un dígit a la vegada. Llavors el que fem és que els encenem de forma seqüencial un darrere de l'altre amb el número que volem que es mostri. Perquè sembli que els quatre dígits estan sempre encesos, la seqüència ha de repetir-se molt ràpidament.
Si parem el refresc o no el fem de forma correcte:

- Un dígit quedarà encès mentre els altres quedaran apagats.
- O bé tots parpellejaran de manera molt evident.
- O veurem un nombre borrós, inestable o amb segments que "ballen".

P5. (1 punt) Per tal de mostrar símbols més complexes, existeixen displays de 16-segments que incrementen el número de leds respecte els que tenim al laboratori. Hem connectat un d'aquests displays al nostre micro PIC18F45K22, segons l'esquema adjunt a continuació. La caiguda de voltatge Vf del led de cada segment és de 2V. El valor de cada resistència és de 125Ω. Volem mostrar el digit '0' amb barra diagonal, tal com es veu a la figura.





pin	segment
RC0	a1
RC1	a2
RC2	b
RC3	c
RC4	d1
RC5	d2
RC6	e
RC7	f
RD0	g1
RD1	g2
RD2	h
RD3	i
RD4	j
RD5	k
RD6	l
RD7	m

5.1 (0,3p) De quin tipus és el display de la figura, ¿càtode o ànode comú?

Es ànode comú, es pot veure com el display te un pin que connecta a Vdd. Per tant, cada segment va a un pin GPIO que posarà un 0 si vol encendre'l (en un càtode comú, veuriem un pin comú de connexió a GND).

5.2 (0,4p) Dóna un codi C que faci les configuracions adequades i mostri el '0' amb barra diagonal.

```
ANSELC=ANSELD=0; // tots els pins en digital
TRISC=TRISD=0; // tots els pins de sortida
LATC=0;  // tots els segments exteriors del '0' encesos
LATD=0xCF;  // només els segments 'k' i 'j' encesos (barra diagonal)
```

5.3 (0,3p) Atenent a les limitacions de corrent dels pins del nostre PIC, podrem encendre el dibuix sense problemes? En cas contrari, indica quin/s canvi/s faries al circuit per solucionar-ho.

Segons especificacions del nostre PIC, sabem que: Maximum output current sunk by any I/O pin: 25mA. I també que Maximum current sunk by all ports (podem suposar el cas -40ºC…+85ºC): 200mA.

Calculem la corrent que entra a cada pin: I = Vresistencia / R = (Vdd – Vf_led) / R. Podem suposar que Vdd=5V, el valor habitual a CI. També s'accepta haver-ho deixat expressat en funció de Vdd, si la resta del raonament és correcte. Suposant Vdd=5V ➔ I = (5-2) / 125 = 24mA.
24mA és vàlid com a corrent per cada pin individual (≤25mA max). Si mirem la suma de totes les corrents, com estem encenent 10 segments, el consum seria de 240mA. Això no seria correcte, ens passem dels 200mA màxims.
El que fariem seria pujar les resistències, per reduir la corrent a cada segment. Cal donar un valor de R, per tant si la I total ha de ser ≤ 200mA ➔ I_led ≤ 20mA ➔ (5-2) / Rnova ≤ 20mA ➔ Rnova ≥ 150Ω.
Si no volguéssim estar just al límit, per tenir marge podriem pujar a 330Ω o 470Ω que son valors usuals al laboratori.

P6. (1 punt) Indica si les afirmacions són certes (C) o falses (F). No cal justificar. Encert +1/8, error -1/16

F	La instrucció LFSR serveix per fer un adreçament indirecte i llegir el valor de qualsevol posició de memòria de dades, deixant-nos el resultat a WREG
F	L'algoritme de scan matrix per llegir un keypad, en cas que estiguem polsant més d'una tecla a la vegada, només podrà llegir una d'elles a causa dels díodes de protecció.
C	Usant un clock de Fosc de 12MHz, el temps d'execució d'una instrucció que provoca un salt de tipus skip i precedeix a una instrucció 'double word' serà de 1µs.

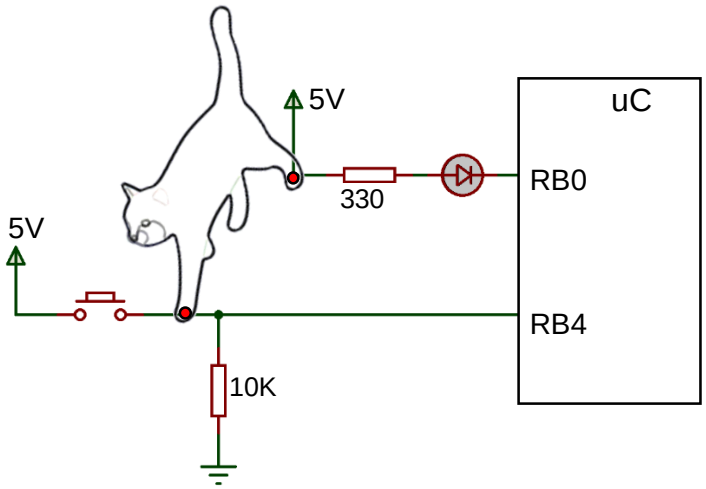
Computer Interfacing (CI) FIB. Primer parcial **03/11/2025**

Cognoms, Nom _____**SOLUCIÓ**_____ **DNI** _____

Tota resposta sense justificar es considerarà nul·la !

F	Usant l'Access Bank podem accedir a qualsevol posició de memòria RAM, però amb l'avantatge que no cal especificar abans el BSR.
C	El byte menys significatiu d'una instrucció, s'emmagatzema sempre en una adreça parell de la memòria de programa.
F	El nostre PIC accedeix als registres de control dels perifèrics amb un model "Isolated I/O", per això la memòria RAM té bancs de propòsit general diferents del banc dels SFR.
C	Les següents instruccions: "setf FF, A" i "setf FF, B" accedeixen a la mateixa posició de memòria si BSR=F.
F	En un pin configurat com a Output, el sentit de la corrent elèctrica mai pot ser de l'exterior cap a l'interior del pin

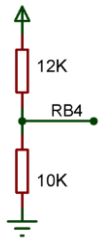
P7. (1,5 punts) Hem fet un programa que consulta per enquesta la pulsació d'un botó connectat al pin RB4. Quan el premem fa encendre un LED que està al pin RB0. L'esquema és el que es veu a la següent figura, però en un moment donat el nostre gat, que camina descuidadament, toca amb una pota el cable de Vdd i amb l'altra el pin RB4. Considera a efectes d'aquest exercici que la resistència entre les potes del gat és de 12K, i que el botó no es pulsa mai.




```
void main() {
  configPIC();
  LATBbits.LATB0=1;
  while (1) {
    if (PORTBbits.RB4) LATBbits.LATB0=0;
  }
}
```

7.1 (1p) Ha alterat el gat el funcionament del nostre programa, fent que s'encengui el led de RB0?

Quan el gat està trepitjant tal com es veu a l'enunciat, és com si tinguéssim al pin de input RB4 un circuit divisor de tensió. Si calculem el voltatge que apareix al punt mig:
V_{RB4} = 5V * (10K / (10K + 12K)) = 2.27V.



Com que un pin del PIC té un V_{IH_min} = 2V, podem assegurar que 2.27V serà interpretat com '1' lògic, i per tant el codi posarà LATB0=0, fent que s'encengui el LED.

7.2 (0,5p) Després que el gat hagi passat i ja es trobi lluny, com estarà el led? Encès o apagat?

Quan el gat hagi marxat, el pull-down de 10K estarà posant un '0' al pin RB4. Però el codi no té cap instrucció que posi de nou el pin RB0 a '1' (per apagar el LED). Per tant, seguirà encès.